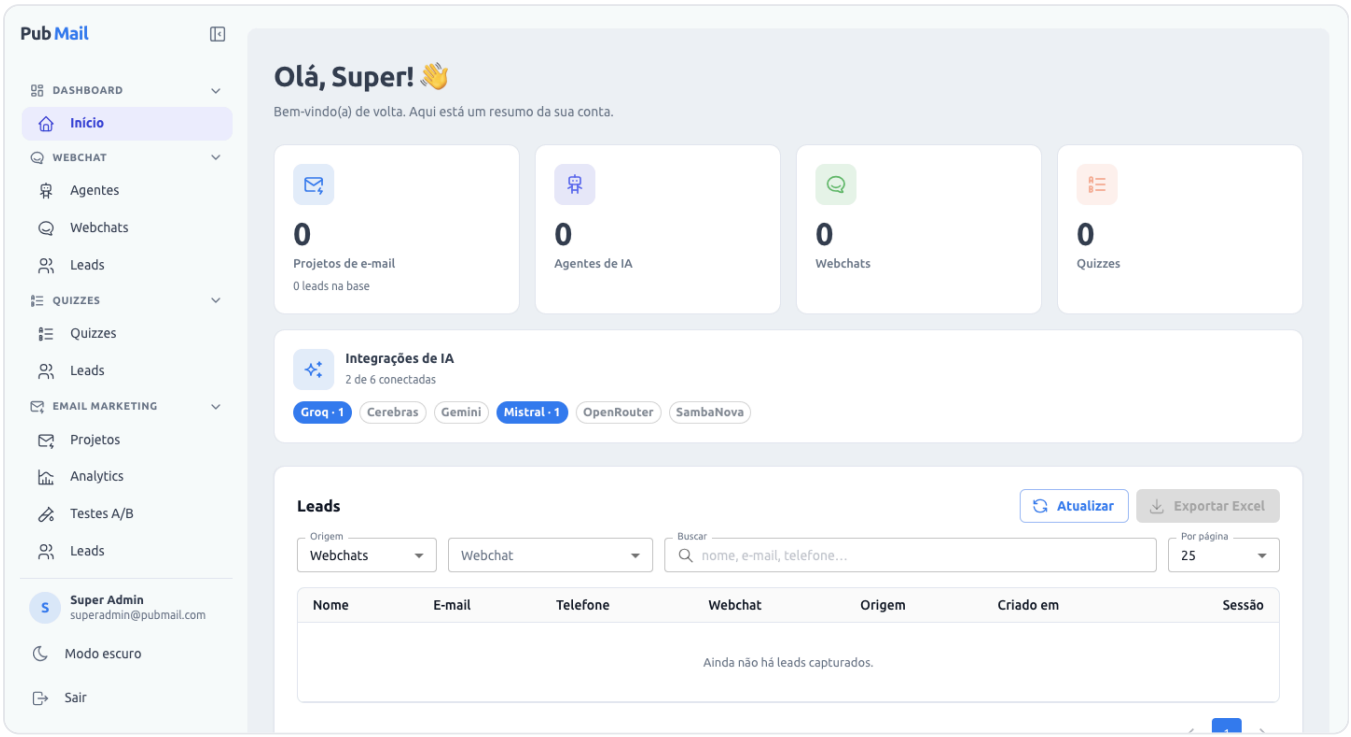
Pub Mail

Documentação da plataforma

Webchats com IA · Quizzes · E-mail marketing · Telegram

Versão de 25/09/2026

Plataforma **multi-tenant** de marketing e automação: **webchats com agentes de IA**, **quizzes**, **e-mail marketing** completo (construtor visual, automações, segmentação, A/B, analytics) e um **módulo Telegram** (bots, DMs, grupos/canais, broadcasts, fluxos, prova social e vendas via PIX). Domínios próprios dos clientes são provisionados automaticamente (nginx + HTTPS Let's Encrypt), e as chamadas de IA fazem rotação inteligente entre múltiplos provedores.



Sumário

- Visão geral
- Módulos
  - Webchat com IA
  - Quizzes
  - Email Marketing
  - Leads e Automações
  - Telegram
  - Administração, Integrações e Conta
- Arquitetura
- Stack
- Estrutura do repositório
- Rodando localmente
- Variáveis de ambiente
- Deploy em produção
- Operação
- Troubleshooting

Visão geral

Cada **organização** (tenant) tem seus usuários, carteira de tokens, agentes, domínios, webchats, quizzes, projetos de e-mail, leads e bots do Telegram — tudo isolado por `organization_id`. Um `SUPER_ADMIN` de plataforma administra organizações e usuários.

O fluxo típico: captar leads (webchat, quiz, formulário no site, Telegram) → nutrir (fluxo inicial, automações, broadcasts, gatilhos) → converter (e-mails, VIP no Telegram com PIX) → medir (analytics, engajamento por lead).

Módulos

Webchat com IA

- **Agentes de IA** com persona/produto (`prompt_mestre`), regras de captura de lead e observações.
- **Domínios próprios** (`chat.suamarca.com`): cliente aponta o DNS, clica "Verificar DNS" e o sistema gera o vhost nginx + certificado SSL sozinho. `ads.txt` (IAB) editável por domínio.
- **Webchats** com slug (`/webchat/duvidas`), personalização visual completa, **botões** (quick replies com classes CSS), **páginas legais** no rodapé, **anúncios** (GPT/ADX, rótulo opcional) e layout responsivo (coluna de 720px no desktop).
- **Splits**: pastas de webchats com **redirecionamento ponderado** (`/webchat/splits/<slug>` sorteia entre os membros por peso, com trava anti-loop).
- **Captura inteligente de leads** na conversa (nome/e-mail/telefone/campos custom), multilíngue, com guardrails no prompt; leads podem ser roteados para um projeto de e-mail.
- **Rotação multi-provedor de IA** (Groq, Cerebras, Mistral, OpenRouter, Gemini, SambaNova) com cooldown/quota por chave em Redis e fallback degradado.

Quizzes

- **Domínios de quiz** (mesmo provisionamento do webchat) e **CRUD de quizzes** com splits ponderados.
- **Construtor visual** (`/quizzes/:id/builder`) com preview real, 3 presets (Clean, Vibrante, Dark Neon), import/export de preset, texto descritivo por pergunta, blocos de conteúdo (texto/imagem/divisor) e classes CSS por botão.
- **Captação de lead** ao final (campos configuráveis) → `quiz_leads` e, se vinculado, o projeto de e-mail.
- **Redirecionamento por resposta** (cada opção pode ter sua URL) ou geral.
- **E-mail imediato ao lead** (HTML/construtor visual, variáveis `{{name}}`/`{{email}}`/`{{phone}}`).
- **Anúncios** (topo) espelhados do webchat e **rastreamento pronto para GTM** (`<a>` reais + eventos `dataLayer`: `quiz_answer`, `quiz_lead`, `quiz_complete`).

Email Marketing

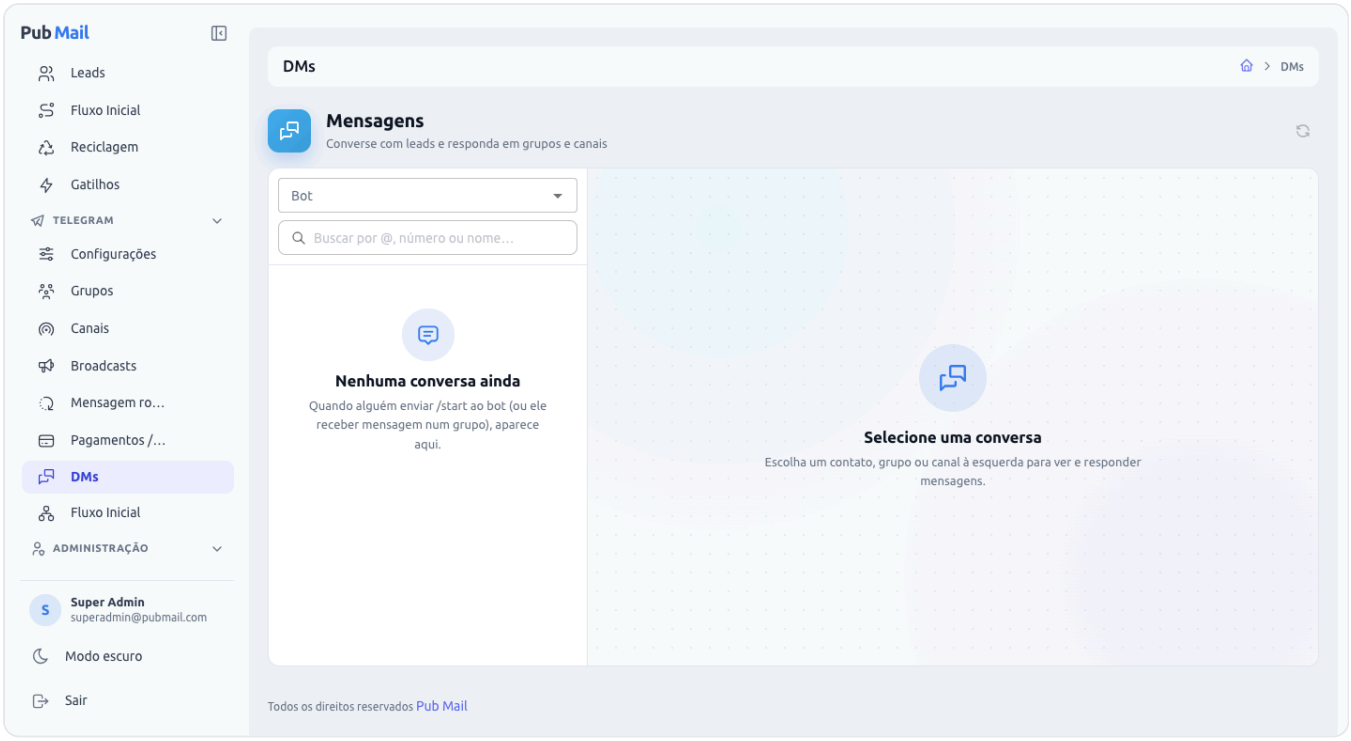
- **Projetos** (listas) com remetente, bloqueio de leads frios e "apenas leads que interagiram no fluxo".

- **Construtor visual por blocos** (texto, imagem, botão rastreado, divisor, espaçador, card, colunas, barra no topo) que gera **HTML mínimo e limpo** para cair na caixa de entrada; **preheader** e **From Name** por template; imagens otimizadas (sharp) servidas direto pelo nginx com cache de 1 ano.
- **Importar templates do Claude**: prompt pronto + planilha `.xlsx/.csv` em formato plano → templates editáveis no construtor.
- **Agendamentos recorrentes** por horário (minuto-do-dia em Brasília), **templates por agendamento**, **disparo manual**, **reenvio para não-abridores**.
- **Segmentação** por regras (abriu/clicou em X dias, taxa de abertura/clique, tags, origem, data de entrada) com contagem ao vivo.
- **Testes A/B** de assunto + template com decisão automática e envio do vencedor ao restante.
- **Formulário de captação v2**: um **único código** copiável (formato WordPress: cola no editor de código e edita no visual) — campos, textos, botão com URL de destino, cores, feedback opcional, tudo configurável e salvo por projeto. Sem e-mail o botão é um link normal; com e-mail capta e redireciona. API pública `POST /email/leads/subscribe/:org/:project` para formulários próprios.
- **Entregabilidade**: validação estrita de e-mail em todas as entradas, higiene automática (conserta/remove inválidos), **webhooks da Resend** (entregue/bounce/spam) com **supressão automática** e estorno de tokens quando nada é enviado.
- **Analytics**: disparos, entregues, aberturas, cliques únicos, bounces, spam, série temporal e engajamento atual da base.

Leads e Automações

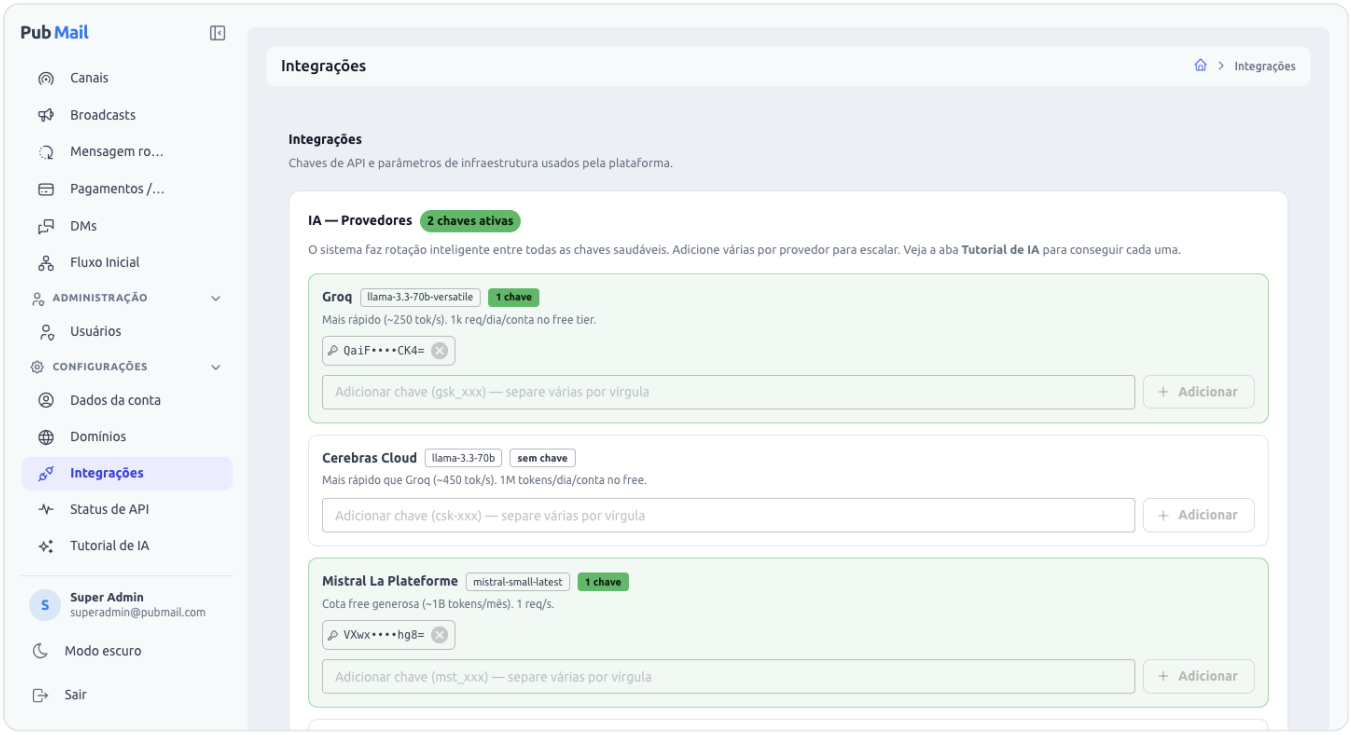
- **Leads** unificados por origem (Webchats, Quizzes, Projetos de e-mail): engajamento por lead, **tags** (edição individual e em massa), filtros server-side, exportação Excel.
- **Fluxo Inicial** (drip por projeto): sequência de e-mails com atrasos, **condições por passo** (só se abriu/clicou o anterior) e **métricas por passo**.
- **Reciclagem (win-back)**: agendamentos que disparam **só para leads frios**.
- **Gatilhos por comportamento**: ao abrir/clicar → adicionar tag ou enviar template, com frequência por lead (uma vez, cooldown, ilimitado).

Telegram



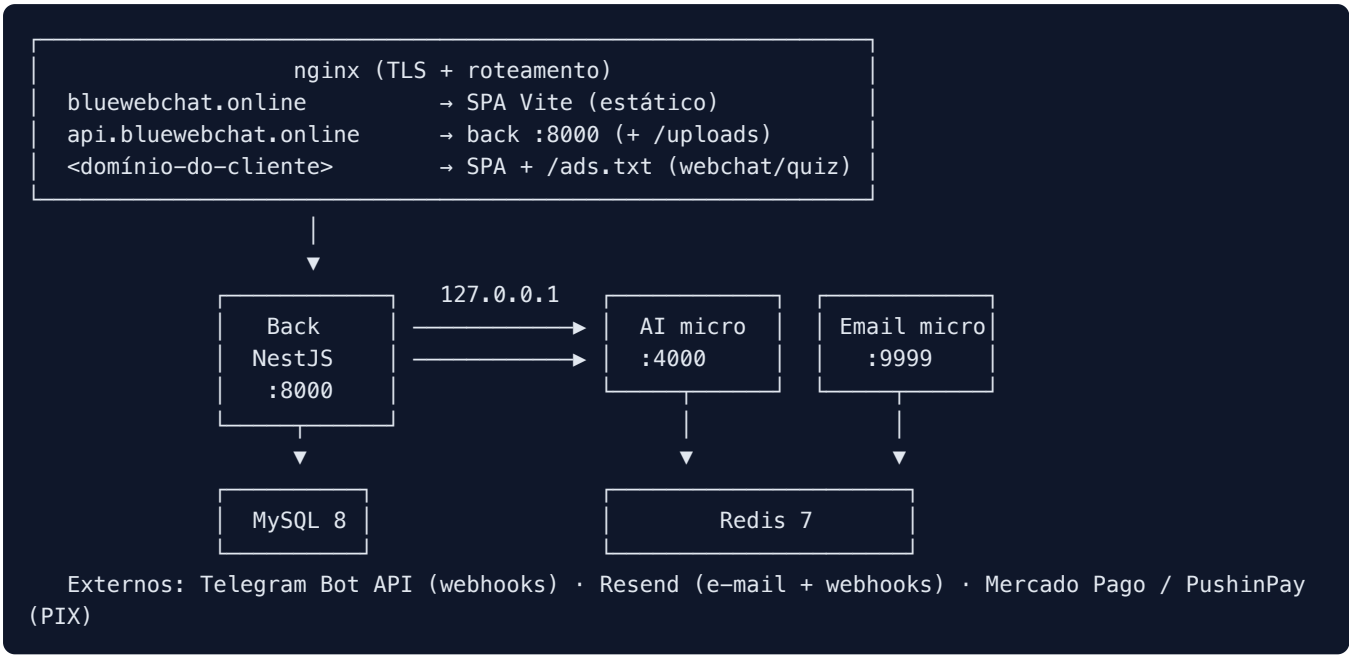
- **Bots** (token do @BotFather) com webhook automático, revalidação, perfil (nome, descrição, comandos) e **aviso de flood/ban** no card.
- **DMs** estilo Telegram (fotos, emojis, anexos, tags do contato, presença), **Grupos** e **Canais** auto-descobertos (convite, foto, contagem de membros, sair).
- **Fluxo Inicial**: construtor visual (reactflow) com blocos arrastáveis — mensagens múltiplas, mídia/áudio (voice via ffmpeg), atraso "digitando", botões (próximo passo, link, grupo, outro bot, **vender plano**), respostas por texto, tags, deep-link de captação (`?start=`).
- **Automações** temporizadas após o `/start` (condição "inativo", tags, botões).
- **Broadcasts** em massa (DMs + grupos + canais, vários bots) com rotação de copies, horários diários, variáveis (`{{nome}}` etc.), botões inline, fila com rate-limit por bot, status por horário e histórico paginado.
- **Mensagem rotativa (prova social)**: uma mensagem que é **editada em loop** ("Fulano acabou de aderir…") em grupos/canais ou nas DMs, sempre como última mensagem, com travas anti-flood (cadência mínima, teto de edições por lead, detecção de lead que saiu).
- **Pagamentos / VIP**: `/vip` → planos → **PIX** (Mercado Pago ou PushinPay, QR + copia-e-cola com botão de copiar) → validação real no gateway → **link de convite de uso único** para o grupo/canal VIP. **Assinaturas por tempo** com lembrete, expiração (remove do VIP) e renovação. Mensagens do checkout totalmente configuráveis. Botão de plano também em broadcasts, automações e fluxo.

Administração, Integrações e Conta



- **Administração → Usuários** (`SUPER_ADMIN`): organizações e usuários cross-org, criação de org + admin, troca de senha, exclusão em cascata. Suporte a **organização oculta** do desenvolvedor (invisível para os demais).
- **Integrações**: chaves de IA por provedor **mascaradas** (adicionar/remover por chave, rotação automática), Resend + Signing Secret do webhook, IP de borda e e-mail do certbot. **Status de API** mostra o estado runtime de cada chave; **Tutorial de IA** ensina a obter as chaves.
- **Conta & Domínios**: dados da conta, domínios de webchat e de quiz.
- **Interface**: tema **claro/escuro**, sidebar em grupos recolhíveis, dashboard com resumo da conta.

Arquitetura



- **Front (Vite/React)**: SPA servida pelo nginx no domínio da plataforma e em todos os domínios de cliente (mesmo `dist/`, roteia por `Host`). O browser **só fala com o back**.
- **Back (NestJS)**: API REST (JWT), multi-tenant, provisiona vhosts + certbot, roda os crons/workers (agendamentos, fluxos, A/B, higiene, Telegram) e recebe os webhooks públicos (Telegram, Resend, gateways de pagamento).
- **AI micro** e **Email micro**: só em loopback; o back é o único cliente deles.
- **MySQL 8** (Prisma) e **Redis** (estado das chaves de IA + filas BullMQ de e-mail).

Stack

Componente	Versão
Node	20.x
Yarn	4.x (corepack, `nodeLinker: node-modules`)
NestJS	11
Prisma	7 (adapter MariaDB)
React / Vite / MUI	19 / 7 / 7
reactflow	11
MySQL	8.0
Redis	7
nginx / Certbot	1.24+ / 2.9+
ffmpeg, libvips (sharp)	dependências de sistema (áudio/imagens)
Ubuntu (prod)	24.04 LTS

Estrutura do repositório

```
pub-mail/
├── back/                      # NestJS API + Prisma
│   ├── prisma/                # schema.prisma + migrations
│   └── src/
│       ├── auth/ admin/       # JWT, roles, administração
│       ├── webchats/ quizzes/ # webchat, domínios, quizzes, splits, anúncios
│       ├── email-marketing/   # projetos, templates, leads, flows, ab, triggers, analytics…
│       ├── cron-jobs/         # agendamentos de e-mail
│       ├── telegram/          # bots, chat, fluxos, automações, broadcasts, rotators, payments
│       └── webhooks/          # Resend + tracking de abertura/clique
├── ai-micro-services/         # roteador multi-provedor de IA
├── email-micro-services/      # dispatcher Resend (BullMQ)
├── front/vite/                # SPA React
│   └── src/views/pages/       # webchat, quiz, email, leads, telegram, admin…
├── docs/images/               # imagens deste README
├── docker-compose.yml         # MySQL + Redis + phpMyAdmin para dev local
└── CLAUDE.md                  # notas operacionais detalhadas (deploy, gotchas, histórico)
```

Rodando localmente

Pré-requisitos

Node 20, Yarn 4 (corepack), Docker (para MySQL/Redis), `ffmpeg` (áudio do Telegram) e `libvips` (sharp) — ambos opcionais em dev.

Setup

```
# 1. Dependências de infra
docker compose up -d            # MySQL :3306, Redis :6379, phpMyAdmin :8081

# 2. Back
cd back && yarn install
yarn prisma migrate deploy && yarn prisma generate
yarn start:dev                  # :8000

# 3. AI micro (outro terminal)
cd ai-micro-services && yarn install && yarn dev   # :4000

# 4. Email micro (outro terminal)
cd email-micro-services && yarn install && yarn dev   # :9999

# 5. Front (outro terminal)
cd front/vite && yarn install && yarn dev   # :3000
```

> Webhooks (Telegram, Resend, gateways) precisam de uma URL pública — em dev use um túnel (ex.: `ngrok http 8000`) e aponte `PUBLIC_API_URL` para ele.

Variáveis de ambiente

Back (NestJS)

Variável	Default	Descrição
`PORT`	8000	Porta HTTP
`DATABASE_HOST/USER/PASS/NAME/PORT`	—	**Conexão usada em runtime** (adapter MariaDB)
`DATABASE_URL`	—	Só para o CLI do Prisma (migrate/generate)
`DATABASE_POOL_LIMIT`	10	**Obrigatório** — o back abre vários pools; com o default (50) estoura o `max_connections` do MySQL
`REDIS_HOST` / `REDIS_PORT`	127.0.0.1 / 6379	Redis
`JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET`	—	Segredos JWT
`IA_SERVICE_URL` / `IA_SERVICE_KEY`	`http://127.0.0.1:4000`	AI micro
`EMAIL_SERVICE_URL` / `EMAIL_SERVICE_KEY`	`http://127.0.0.1:9999`	Email micro
`PUBLIC_API_URL`	`https://api.bluewebchat.online`	URL pública do back (webhooks do Telegram/pagamentos, formulário de captação)
`WEBCHAT_FRONT_DIST`	`/opt/pub-mail/front/vite/dist`	Build do front servido nos domínios de cliente
`WEBCHAT_BACKEND_INTERNAL_URL`	`http://127.0.0.1:8000`	Usado nos vhosts gerados para `/ads.txt`
`NGINX_SITES_AVAILABLE` / `NGINX_SITES_ENABLED`	`/etc/nginx/...`	Onde o back escreve os vhosts
`CERTBOT_BIN` / `CERTBOT_MODE`	`certbot` / `nginx`	Provisionamento SSL
`HTTP_BODY_LIMIT`, `UPLOAD_MAX_FILE_SIZE_BYTES`	—	Limites de upload

Configurações de plataforma (chaves de IA, Resend, webhook secret, IP de borda, e-mail do certbot) ficam no banco (`system_settings`) e são editadas em **Integrações**.

AI micro

Variável	Default	Descrição
`PORT`	4000	
`IA_SERVICE_KEY`	—	Validado em `x-api-key`
`REDIS_HOST` / `REDIS_PORT`	127.0.0.1 / 6379	Estado das chaves
`OPENROUTER_REFERER`	—	Header recomendado pelo OpenRouter

Email micro

Variável	Default	Descrição
`PORT`	9999	
`EMAIL_SERVICE_KEY`	—	Validado em `x-api-key`
`REDIS_HOST` / `REDIS_PORT`	127.0.0.1 / 6379	Filas BullMQ
`NEST_WEBHOOK_URL`	—	Callback interno para o back (resultado do disparo)
`BASE_URL_API`	—	URL pública do back (pixel de abertura / clique rastreado)

Front (Vite)

Variável	Obrigatória	Descrição
`VITE_APP_BASE_NAME`	**sim (`/`)**	Sem isso os assets quebram (tela branca)
`VITE_API_URL`	sim	URL pública do back — o browser só fala com ele

Deploy em produção

> **Set/2026:** a instalação na AWS (EC2 via SSM) está sendo **desativada**. O próximo ambiente é um **servidor novo com banco limpo**. O runbook abaixo é o caminho oficial de instalação do zero; o histórico da AWS ficou só no `CLAUDE.md`.

Deploy **nativo** (sem Docker): Ubuntu 24.04, Node 20 + Yarn 4 (corepack), MySQL 8, Redis 7, nginx + certbot, `ffmpeg` e `libvips`. Serviços como `systemd`. O código fica em `/opt/pub-mail`.

1. Pacotes

```
apt update && apt install -y nginx mysql-server redis-server certbot python3-certbot-nginx ffmpeg
libvips42 build-essential git
curl -fsSL https://deb.nodesource.com/setup_20.x | bash - && apt install -y nodejs && corepack
enable
```

2. MySQL + Redis

```
mysql -e "CREATE DATABASE pubmail CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -e "CREATE USER 'pubmail'@'127.0.0.1' IDENTIFIED WITH mysql_native_password BY '<DB_PASS>';
GRANT ALL ON pubmail.* TO 'pubmail'@'127.0.0.1'; FLUSH PRIVILEGES;"
# recomendado: innodb_buffer_pool_size ~25% da RAM; redis: maxmemory + noeviction
```

> O banco **precisa** ser `utf8mb4_unicode_ci` — as migrations criam tabelas com esse collation e as FKs falham se o default do servidor for outro.

3. Código e `.env`

```
git clone <repo> /opt/pub-mail
cd /opt/pub-mail && for d in back ai-micro-services email-micro-services front/vite; do cp
$d/.env.example $d/.env; done
# preencha cada .env (os .env NÃO são versionados) → ver "Variáveis de ambiente"; gere os
segredos com: openssl rand -hex 32
```

Pontos que sempre dão problema (veja a tabela de variáveis): `DATABASE_HOST/USER/PASS/NAME/PORT` (runtime) **e** `DATABASE_URL` (CLI do Prisma); `DATABASE_POOL_LIMIT=10`; `WEBCHAT_BACKEND_INTERNAL_URL=http://127.0.0.1:8000`; `PUBLIC_API_URL`, `BASE_URL_API` e `NEST_WEBHOOK_URL` com a URL pública nova do back; `VITE_APP_BASE_NAME=/` e `VITE_API_URL` no front.

4. Build + banco + seed

```
cd /opt/pub-mail/back && yarn install && yarn prisma migrate deploy && yarn prisma generate &&
yarn nest build
# usuário de plataforma (SUPER_ADMIN, organização oculta) + IP de borda + e-mail do certbot — sem
senha no código:
SEED_ADMIN_EMAIL=dev@suaempresa.com SEED_ADMIN_PASSWORD='<senha forte>' SEED_EDGE_IP=<IPv4 do
servidor> SEED_CERTBOT_EMAIL=ops@suaempresa.com \
  node scripts/seed-platform.mjs | mysql -u pubmail -p'<DB_PASS>' -h 127.0.0.1 pubmail

cd /opt/pub-mail/ai-micro-services && yarn install
cd /opt/pub-mail/email-micro-services && yarn install
cd /opt/pub-mail/front/vite && yarn install && NODE_OPTIONS=--max-old-space-size=4096 yarn build
```

O seed é **idempotente** (pode rodar de novo para trocar a senha). Cada organização cadastra **as próprias chaves** (IA, Resend, webhook) em *Configurações → Integrações* — nada é compartilhado entre organizações. IP de borda e e-mail do certbot são da plataforma e só o `SUPER_ADMIN` edita.

5. Serviços (`systemd`)

Três units — `pubmail-back` (`node dist/src/main.js`, porta 8000, **roda como root** para escrever vhosts do nginx e rodar o certbot), `pubmail-ai` (`node server.js`, 4000) e `pubmail-email` (9999) — com `WorkingDirectory` na pasta de cada serviço, `EnvironmentFile=<pasta>/.env`, `Restart=always`. Os micros ficam **só em loopback** (o email-micro não tem autenticação no `/send`).

6. nginx + SSL

- Vhost do **front** (domínio da plataforma): `root /opt/pub-mail/front/vite/dist` + `try_files $uri /index.html`.
- Vhost da **API** (`api.<domínio>`): `location ^~ /uploads/ { alias /opt/pub-mail/back/uploads/; expires 365d; add_header Cache-Control "public, immutable"; }` **antes** do `location / { proxy_pass http://127.0.0.1:8000; }`.
- `000-default-deny` (`default_server` retornando 444) para hosts não cadastrados.
- `certbot --nginx -d <front> -d api.<front>`. Domínios de cliente são provisionados pelo próprio sistema (vhost + certbot automáticos ao clicar "Verificar DNS").
- Firewall: só **80/443** (e o acesso administrativo que o time de infra definir).

7. Após subir

Webhooks a reapontar para a URL nova: **Telegram** (o sistema re-seta o webhook ao revalidar cada bot em *Telegram → Configurações*), **Resend** (`https://api.<domínio>/webhooks/resend`, por organização), **gateways PIX** (*Pagamentos → Gateway*). Depois, em cada organização, cadastrar as chaves em Integrações.

Atualizações (depois da instalação)

```
# Front: build em pasta temporária + swap atômico (se falhar, o site fica intacto)
cd /opt/pub-mail/front/vite && rm -rf dist_new && NODE_OPTIONS=--max-old-space-size=4096 yarn
build --outDir dist_new
[ -f dist_new/index.html ] && { rm -rf dist_prev; mv dist dist_prev; mv dist_new dist; }

# Back: migrate (se houve migration) → build → reinicia SÓ se o build passou sem erro de tipo
cd /opt/pub-mail/back && yarn prisma migrate deploy && yarn prisma generate
yarn nest build 2>&1 | tee /tmp/build.log; grep -qE "error TS|Found [0-9]+ error" /tmp/build.log
|| systemctl restart pubmail-back
```

> `nest build` **gera `dist/` mesmo com erro de TypeScript** — por isso o `grep`. Nunca reinicie um build com erro.

Operação

```
journalctl -u pubmail-back -f                    # logs (ou pubmail-ai / pubmail-email)
journalctl -u pubmail-back --since "1 hour ago" -p err
certbot renew                                    # renovação (o timer já faz sozinho)
mysqldump -u pubmail -p pubmail | gzip > backup.sql.gz
```

- **Regenerar vhosts** de todos os domínios: `POST /webchat-domains/regenerate-nginx` (JWT da org) e, se necessário, `certbot --nginx -d <domínio>`.
- **Telegram em flood-wait**: o card do bot mostra "Em pausa" com a hora de retorno; os runners respeitam o `retry_after` sozinhos.

- **Isolamento por organização**: chaves de IA/Resend/webhook ficam em `organization_settings` (uma linha por org); `system_settings` só tem infra da plataforma. Ao criar endpoints novos, **sempre** filtrar por `organization_id` (e `hidden:false` em listagens cross-org).

Troubleshooting

- **Tela branca no front** — `VITE_APP_BASE_NAME=/` precisa existir **antes** do `yarn build`.
- **"Too many connections" no MySQL** — falta `DATABASE_POOL_LIMIT=10` no `back/.env`.
- **`/ads.txt` quebrado em domínio de cliente** — `WEBCHAT_BACKEND_INTERNAL_URL` deve apontar para `:8000`.
- **"Verificar DNS" falha com o DNS correto** — resolver com cache: use `1.1.1.1` / `8.8.8.8` no `systemd-resolved` e reinicie o back.
- **certbot "Another instance is already running"** — `pkill -f certbot && rm -f /var/lib/letsencrypt/.certbot.lock`.
- **Disparo com 0 enviados** — e-mail inválido derrubava o lote (corrigido: validação + fallback individual + estorno de tokens). Verifique `error_message` no envio.
- **Bot do Telegram não responde** — veja o card em Telegram → Configurações: token revogado (permanente, recadastrar) ou flood-wait (temporário, volta sozinho).
- **Yarn 4 + Prisma** — sempre `nodeLinker: node-modules` no `.yarnrc.yml`.

Licença: privada / proprietária.